In [ ]:
```
# Q2
```

In [2]:
```
def newton_raphson(f, f_prime, initial_guess, tolerance=1e-6, max_iterations=100):

    x = initial_guess
    for i in range(max_iterations):
        f_x = f(x)
        f_prime_x = f_prime(x)

        if f_prime_x == 0:
            print("Derivative is zero.Can't Use this method")
            return None

        x_new = x - f_x / f_prime_x

        if abs(x_new - x) < tolerance:
            return x_new

        x = x_new

    print("Maximum iterations reached. Method may not have converged.")
    return x
```

In [ ]:
```
# Q2 III. a)
```

In [5]:
```
import numpy as np

def f(x):
    return x**2 - np.exp(-x) - 2 * np.sin(x)

def f_prime(x):
    return 2*x + np.exp(-x) - 2 * np.cos(x)

def newton_raphson(f, f_prime, initial_guess, tolerance=1e-6, max_iterations=100):
    x = initial_guess
    for i in range(max_iterations):
        f_x = f(x)
        f_prime_x = f_prime(x)

        if f_prime_x == 0:
            print("Derivative is zero. Can't use this method.")
            return None

        x_new = x - f_x / f_prime_x

        if abs(x_new - x) < tolerance:
            return x_new

        x = x_new

    print("Maximum iterations reached. Method may not have converged.")
    return x
```

```
root = newton_raphson(f, f_prime, 2)
print("Root of f(x):", root)
```

```
Root of f(x): 1.4895886408077366
```

In [6]:
```
# Q2 III. b)
```

In [12]:
```
import numpy as np

def f(t):
    return 10 * t - 4.9 * t**2

def f_prime(t):  # 1st derivative of function
    return 10-(4.9*2*t)

def newton_raphson(f, f_prime, initial_guess, tolerance=1e-6, max_iterations=100):
    x = initial_guess
    for i in range(max_iterations):
        f_x = f(x)
        f_prime_x = f_prime(x)

        if f_prime_x == 0:
            print("Derivative is zero. Can't use this method.")
            return None

        x_new = x - f_x / f_prime_x

        if abs(x_new - x) < tolerance: # magnitude of difference between new x and
            return x_new

        x = x_new

    print("Maximum iterations reached. Method may not have converged.")
    return x

root = newton_raphson(f, f_prime, 2)
print("Time when projectile returns to initial height:", root, "seconds")
```

```
Time when projectile returns to initial height: 2.0408163265306736 seconds
```

In [13]:
```
# Q2 III. c)
```

In [18]:
```
def f(x):
    return x**2 - 3

def f_prime(x):  # 1st derivative of function
    return 2*x

def newton_raphson(f, f_prime, initial_guess, tolerance=1e-6, max_iterations=100):
    x = initial_guess
    for i in range(max_iterations):
        f_x = f(x)
        f_prime_x = f_prime(x)

        if f_prime_x == 0:
            print("Derivative is zero. Can't use this method.")
```

```
            return None

        x_new = x - f_x / f_prime_x

        if abs(x_new - x) < tolerance: # magnitude of difference between new x and
            return x_new

        x = x_new

    print("Maximum iterations reached. Method may not have converged.")
    return x

root = newton_raphson(f, f_prime, 2)
print("Approximate sqrt(3):", root)
```

```
Approximate sqrt(3): 1.7320508075688772
```

In [ ]:
```
# Q2 III. d)
```

In [20]:
```
import numpy as np

def f(L):
    return 2 * np.pi * np.sqrt(L / 9.8) - 10

def f_prime(L):
    return np.pi / np.sqrt(9.8 * L)

def newton_raphson(f, f_prime, initial_guess, tolerance=1e-6, max_iterations=100):
    x = initial_guess
    for i in range(max_iterations):
        f_x = f(x)
        f_prime_x = f_prime(x)

        if f_prime_x == 0:
            print("Derivative is zero. Can't use this method.")
            return None

        x_new = x - f_x / f_prime_x

        if abs(x_new - x) < tolerance: # magnitude of difference between new x and
            return x_new

        x = x_new

    print("Maximum iterations reached. Method may not have converged.")
    return x

root = newton_raphson(f, f_prime, 25)
print("Length L for T = 10 seconds:", root, "meters")
```

```
Length L for T = 10 seconds: 24.823689992372756 meters
```

In [ ]:
```
# Q2 III. e)
```

In [21]:
```
import numpy as np
```

```python
def f(t):
    return 5 * (1 - np.exp(-t / 10)) - 2.5

def f_prime(t):
    return 0.5 * np.exp(-t / 10)

def newton_raphson(f, f_prime, initial_guess, tolerance=1e-6, max_iterations=100):
    x = initial_guess
    for i in range(max_iterations):
        f_x = f(x)
        f_prime_x = f_prime(x)

        if f_prime_x == 0:
            print("Derivative is zero. Can't use this method.")
            return None

        x_new = x - f_x / f_prime_x

        if abs(x_new - x) < tolerance: # magnitude of difference between new x and
            return x_new

        x = x_new

    print("Maximum iterations reached. Method may not have converged.")
    return x

root = newton_raphson(f, f_prime, 10)
print("Time at which Vc(t) = 2.5 V:", root, "seconds")
```

```
Time at which Vc(t) = 2.5 V: 6.931471805599453 seconds
```

In [ ]: